

Unity extOSC

OSC 실시간 제어 강의

설치된 extOSC 1.21.0 패키지 기준으로 OSC 기초, Unity 컴포넌트 사용법, 코드 실습, 디버깅, VTuber/라이브 스튜디오형 포트폴리오 설계를 한 번에 정리한 자료입니다.



Transmitter



Receiver



Message



Bundle

1교시 - OSC 기초

주소, 타입 태그, 인자, UDP, Message와 Bundle의 의미를 잡습니다.

2교시 - extOSC 설치와 구조

현재 프로젝트 설치 상태, Package Manager, Samples, 메뉴와 컴포넌트를 확인합니다.

3교시 - 송수신 실습

Transmitter, Receiver, Bind, OSCMessage, OSCValue를 코드와 Inspector 양쪽으로 익힙니다.

4교시 - 현장 기능

Mapping, Event, Informer, Reflection, UI Controls, Ping, Console, Debug를 연결합니다.

5교시 - 안정성

포트, IP, UTF8, 메시지 폭주, Drown Detection, 테스트 체크리스트를 정리합니다.

6교시 - 포트폴리오

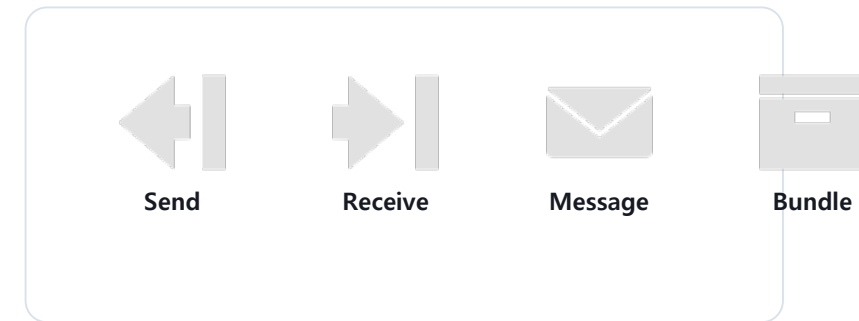
Unity 라이브 스튜디오 제어 패널, VMC 응용, OBS 브릿지, README 주소 계약서를 설계합니다.

학습 원칙

모든 기능을 외우는 것보다 '주소 설계 -> 송수신 -> 검증 -> 운영 UI -> 성능 측정' 순서로 손에 붙이는 것이 중요합니다.

핵심 개념

- OSC는 실시간 미디어 장비, 앱, 엔진 사이에서 작은 제어 메시지를 주고받기 위해 만들어진 메시지 기반 프로토콜입니다.
- extOSC는 Unity에서 OSC 메시지를 더 쉽게 보내고 받도록 해주는 패키지입니다. 코드 방식과 컴포넌트 방식 둘 다 제공합니다.
- VTuber/라이브 스튜디오형 포트폴리오에서는 카메라, 조명, 표정, 효과, 무대 프리셋을 외부 패널에서 즉시 제어하는 능력을 보여줄 수 있습니다.
- 단, 이 자료는 특정 기업이 extOSC를 쓴다고 단정하지 않습니다. 포트폴리오 역량 강화를 위한 실무형 학습 설계입니다.



비유

OSC는 무대 감독이 무전기로 짧게 보내는 큐 사인에 가깝습니다. 긴 파일을 보내는 게 아니라 '조명 2번 80%', '카메라 컷 3번' 같은 신호를 빠르게 보냅니다.

실제 예시

태블릿 슬라이더를 움직이면 `/stage/light/key/intensity 0.75` 메시지가 Unity로 들어오고, Unity는 Directional Light의 intensity를 바꿉니다.

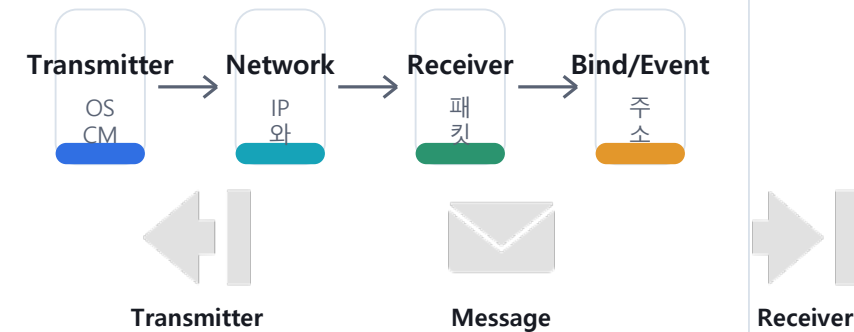
바로 해볼 것

이 PDF를 읽을 때마다 '주소, 값, 받는 오브젝트' 세 가지를 표시해보세요. OSC 설계는 이 세 가지만 잡아도 절반은 풀립니다.

핵심 개념

- 프로젝트 `Packages/manifest.json`에는 `com.iam1337.extosc`가 Git URL `https://github.com/iam1337/extOSC.git#upm`로 등록되어 있습니다.
- `Packages/packages-lock.json` 기준으로 source는 git, hash는 `b7c2bfa81633cbcbc8cc4312e15cb5fbd0ed7d1d`입니다.
- 패키지 캐시의 `package.json` 기준 extOSC 버전은 `1.21.0`이고 Unity 최소 버전 표기는 `2018.1`입니다.
- 현재 프로젝트에는 URP 17.3.0도 들어와 있어 OSC 제어와 렌더 최적화 포트폴리오를 같은 프로젝트 안에서 연결하기 좋습니다.

extOSC 안에서 데이터가 움직이는 길



비유

manifest는 장비 목록, packages-lock은 실제 장비의 시리얼 넘버 기록입니다. '뭘 쓰기로 했는지'와 '정확히 어떤 버전이 들어왔는지'를 나눠서 확인합니다.

실제 예시

면접이나 포트폴리오 README에는 'extOSC 1.21.0, Git UPM dependency, Unity URP 17.3.0 프로젝트에서 검증'처럼 적을 수 있습니다.

바로 해볼 것

Unity에서 Window > Package Manager를 열고

핵심 개념

- OSC Packet의 내용은 OSC Message 또는 OSC Bundle 중 하나입니다.
- OSC Message는 Address Pattern, Type Tag String, Arguments로 구성됩니다.
- Address Pattern은 `/`로 시작하는 문자열입니다. 예: `/camera/fov`, `/avatar/face/smile`.
- Type Tag String은 `;`로 시작하며 뒤에 값 타입이 붙습니다. 예: `,f`는 float 하나, `,sf`는 string과 float입니다.



비유: 주소는 택배 송장, Type tag는 상자 라벨, Arguments는 상자 안 물건입니다. 받는 쪽은 주소가 맞고 라벨이 맞을 때만 안전하게 꺼내 씁니다.

비유

주소는 택배 송장, 타입 태그는 물건 라벨, 값은 실제 물건입니다. 송장과 라벨이 틀리면 받는 쪽에서 안전하게 처리할 수 없습니다.

실제 예시

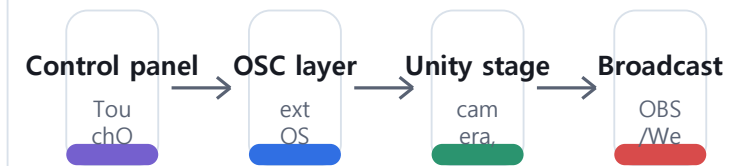
``/stage/fog/density ,f 0.35``는 'stage/fog/density 주소로 float 값 0.35를 보낸다'는 뜻입니다.

바로 해볼 것

프로젝트에서 제어하고 싶은 항목 5개를 골라 ``/stage/...`, `/avatar/...`, `/camera/...`` 주소로 먼저 이름만 만들어보세요.

핵심 개념

- OSC 스펙은 전송 방식에 독립적이지만, UDP datagram으로 자연스럽게 표현될 수 있다고 설명합니다.
- extOSC의 일반 PC용 백엔드는 `System.Net.Sockets.UdpClient`를 사용해 송수신합니다.
- UDP는 빠르고 가볍지만, 메시지 도착 보장이나 순서 보장은 애플리케이션 설계에서 따로 고려해야 합니다.
- 그래서 OSC 제어값은 '최신 상태가 중요하고, 다음 프레임에 또 보낼 수 있는 값'에 잘 어울립니다.



주의

이 구조는 포트폴리오 설계 제안입니다. 특정 기업이 extOSC나 VMC, OBS WebSocket을 사용한다고 단정하지 않습니다.

비유

UDP는 엽서입니다. 빠르게 던질 수 있지만 수신 확인 도장이 자동으로 오지는 않습니다. 중요한 명령은 ACK나 Ping 같은 보조 설계가 필요합니다.

실제 예시

슬라이더로 조명 밝기를 매 프레임 보내는 것은 괜찮습니다. 하지만 '녹화 시작' 같은 명령은 중복 방지나 응답 확인을 설계하는 편이 좋습니다.

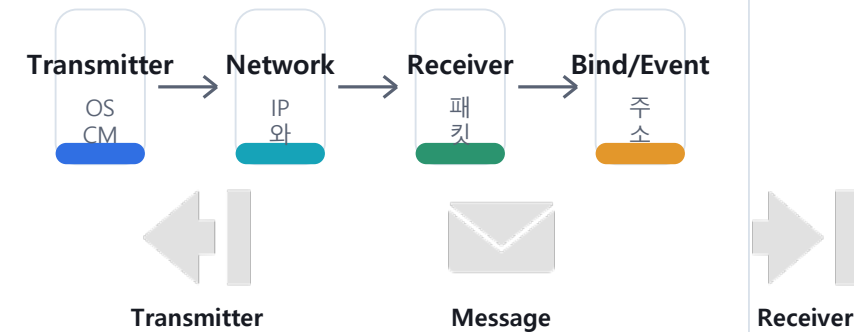
바로 해볼 것

명령을 '상태값', '순간 트리거', '반드시 확인할 명령' 세 그룹으로 나눠보세요.

핵심 개념

- 핵심 클래스는 `OSCTransmitter`, `OSCSReceiver`, `OSCMessage`, `OSCValue`, `OSCBundle`입니다.
- 코드 없이 쓰는 컴포넌트 계층은 Receiver Event, Receiver Reflection, Transmitter Informer, UI Control로 나뉩니다.
- 에디터 도구는 Tools > extOSC > OSC Console, OSC Debug, OSC Mapping 메뉴로 열 수 있습니다.
- 패키지 샘플은 Getting Started부터 Marshalling까지 13개가 제공됩니다.

extOSC 안에서 데이터가 움직이는 길



비유

extOSC는 우체국 세트입니다. Transmitter는 발송 창구, Receiver는 수신 창구, Message는 편지, Bundle은 여러 편지를 묶은 봉투입니다.

실제 예시

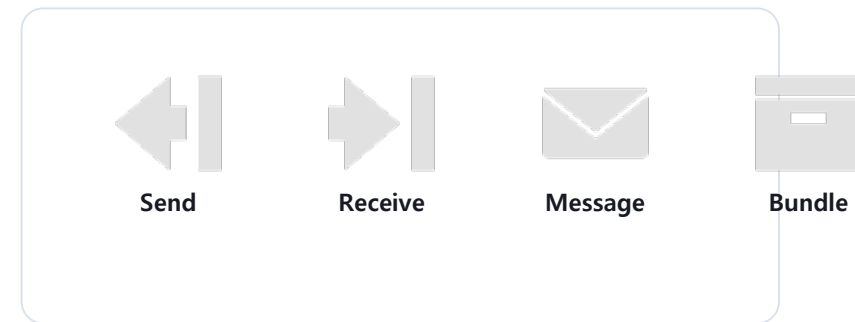
조명 밝기 슬라이더는 Transmitter Informer가 값을 메시지로 만들고, 받는 씬에서는 Receiver Event가 Light.intensity 변경 함수를 호출합니다.

바로 해볼 것

Package Manager에서 extOSC Samples를 Import한 뒤

핵심 개념

- 공식 README는 Unity Package Manager의 Add package from git URL 방식과 OpenUPM 설치 방식을 안내합니다.
- 현재 프로젝트는 Git URL 방식입니다. 이 방식은 특정 브랜치나 태그를 직접 참조하기 쉽습니다.
- OpenUPM 방식은 scoped registry를 추가해 패키지명과 버전으로 관리하는 흐름에 가깝습니다.
- 포트폴리오 프로젝트에서는 lock 파일을 함께 커밋해 재현 가능한 설치 상태를 남기는 것이 좋습니다.



비유

Git URL은 제작자 창고에서 직접 가져오는 방식, OpenUPM은 부품 매장에서 품번으로 주문하는 방식에 가깝습니다.

실제 예시

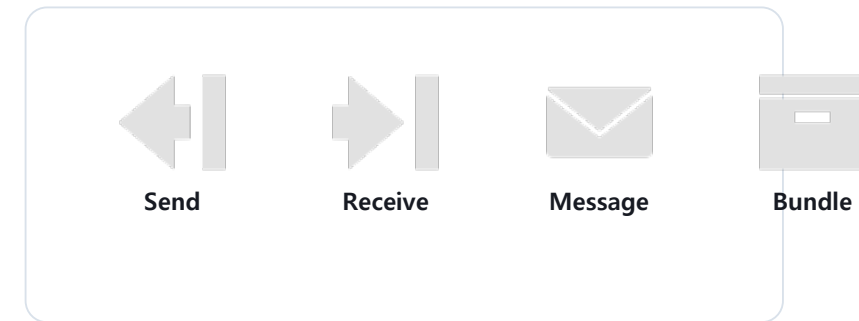
README에 ``com.iam1337.extosc:`
`https://github.com/iam1337/extOSC.git#upm``와 lock hash를 적어두면 다른 사람이 환경을 맞추기 쉽습니다.

바로 해볼 것

``Packages/manifest.json``과 ``packages-lock.json``을 열고 extOSC 항목을 캡처해서 포트폴리오 문서의 환경 섹션에 넣어보세요.

핵심 개념

- Unity 패키지는 `package.json`의 `samples` 배열을 통해 Package Manager에서 Import 가능한 예제를 제공합니다.
- extOSC는 Getting Started, Value Types, Events, Informers, Events And Informers, Mapping, Scripting, UI, Address Masks, Ping, Array, Match Pattern, Marshalling 샘플을 포함합니다.
- 처음부터 모든 샘플을 외우기보다 메시지 송수신 -> 값 타입 -> UI -> Mapping -> 검증/고급 순서로 보는 것이 좋습니다.
- 샘플 코드는 포트폴리오용 코드 작성 패턴의 출발점으로 사용할 수 있습니다.



비유

샘플은 요리책의 기본 레시피입니다. 그대로 만들고, 재료 하나씩 바꿔보고, 마지막에 자기 메뉴로 바꾸는 순서가 가장 빠릅니다.

실제 예시

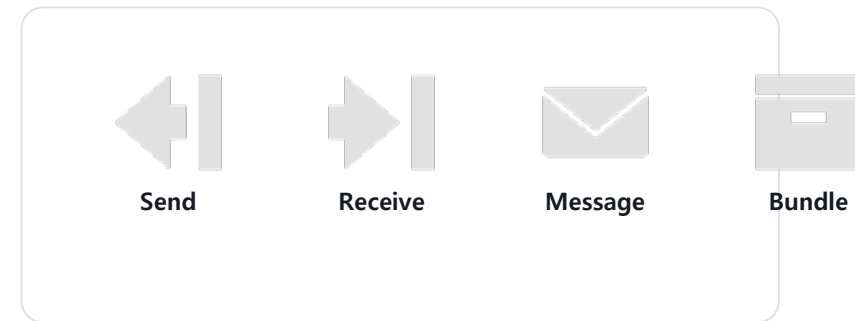
Getting Started에서 `/example/1` 메시지를 보내고 받은 뒤, 같은 구조를 `/stage/light/key/intensity`로 바꿔보는 식입니다.

바로 해볼 것

샘플 13개를 모두 Import하지 않아도 됩니다. 우선 Getting Started, Scripting, UI, Mapping, Ping 다섯 개부터 가져오세요.

핵심 개념

- `OSCTransmitter`는 OSC 패킷을 만들어 RemoteHost와 RemotePort로 보냅니다.
- 기본 remote host는 `127.0.0.1`, 기본 remote port는 `7000`입니다.
- LocalPortMode에는 FromRemotePort, FromReceiver, Random, Custom 같은 선택지가 있습니다.
- `UseBundle`이 켜져 있으면 프레임 안에서 모은 메시지를 번들로 묶어 보낼 수 있습니다.



비유

Transmitter는 송신기입니다. 상대 IP와 포트는 전화번호, 메시지 주소는 내선번호, 값은 통화 내용입니다.

실제 예시

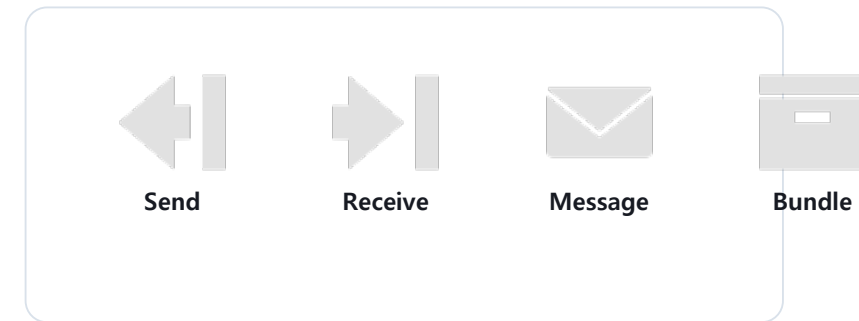
Unity가 외부 앱에 현재 카메라 FOV를 알려야 한다면 `/unity/camera/fov 45.0` 메시지를 Transmitter로 보냅니다.

바로 해볼 것

빈 GameObject에 OSCTransmitter를 붙이고 RemoteHost를 `127.0.0.1`, RemotePort를 `7001`로 맞추고 테스트 메시지를 보내보세요.

핵심 개념

- 'OSCReceiver'는 LocalHost와 LocalPort를 열어 들어오는 OSC 패킷을 받습니다.
- 기본 local port는 '7001'입니다.
- 받은 패킷은 내부 큐에 쌓이고 Unity 'Update()'에서 바인딩과 맵핑을 거쳐 콜백을 실행합니다.
- 'Bind(address, callback)'으로 특정 주소와 처리 함수를 연결합니다.



비유

Receiver는 우편함입니다. 포트는 우편함 번호이고, Bind는 '이 주소로 온 편지는 이 담당자에게 넘긴다'는 분류 규칙입니다.

실제 예시

'Receiver.Bind("/stage/light/key/intensity", OnLightIntensity)'로 조명 밝기 메시지만 받아서 처리할 수 있습니다.

바로 해볼 것

OSCReceiver의 LocalPort를 7001로 두고, 같은 포트를 다른 앱이 이미 쓰면 어떤 Console 에러가 나는지 확인해보세요.

핵심 개념

- `OSCMessages`는 Address와 `List<OSCValue>`를 가집니다.
- `OSCValue`는 Int, Long, Bool, Float, Double, String, Null, Impulse, Char, Color, Blob, TimeTag, Midi, Array 타입을 제공합니다.
- 타입별 정적 생성 메서드가 있어 `OSCValue.Float(0.8f)`처럼 값을 만들 수 있습니다.
- `ToFloat`, `ToInt`, `ToBlob`, `ToArray` 같은 확장 메서드로 안전하게 값을 읽을 수 있습니다.



비유: 주소는 택배 송장, Type tag는 상자 라벨, Arguments는 상자 안 물건입니다. 받는 쪽은 주소가 맞고 라벨이 맞을 때만 안전하게 꺼내 씁니다.

비유

Message는 주문서, Value는 주문 항목입니다. 항목마다 '수량', '색상', '메모'처럼 타입이 다르기 때문에 라벨을 정확히 붙입니다.

실제 예시

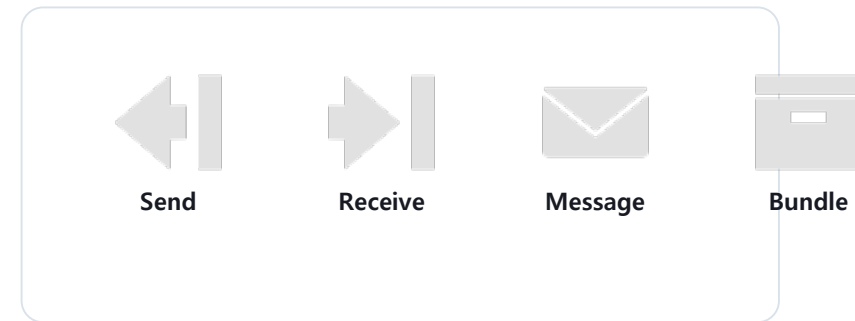
```
`new OSCMessage("/avatar/blink",  
OSCValue.Float(1f))`는 깜빡임 강도 하나를 담은  
메시지입니다.
```

바로 해볼 것

float, bool, string 세 타입만 먼저 사용해보고,
Console에서 출력되는 메시지 문자열을 비교해보세요.

핵심 개념

- OSC Bundle은 여러 OSC Packet을 하나의 묶음으로 담는 구조입니다.
- OSC 스펙상 OSC Packet의 내용은 Message 또는 Bundle입니다.
- extOSC README는 자동 번들을 최적화 목적으로 설명합니다.
- Transmitter의 `UseBundle`은 여러 메시지를 한 프레임에서 묶어 보내는 흐름에 사용됩니다.



비유

Message가 편지 한 장이라면 Bundle은 여러 편지를 한 봉투에 넣는 것입니다. 우체국 창구 방문 횟수를 줄이는 느낌입니다.

실제 예시

한 프레임에 `/face/smile`, `/face/blink`, `/stage/light`를 모두 보낼 때 Bundle로 묶으면 수신 쪽에서 한 묶음으로 받을 수 있습니다.

바로 해볼 것

동시에 보내는 값이 3개 이상일 때 UseBundle을 켜고 OSC Console에서 Message와 Bundle 로그 차이를 비교하세요.

핵심 개념

- extOSC는 주소에 '*' 마스크를 사용할 수 있습니다.
- 예제에는 `/example/9/\*`가 `/example/9/first`, `/example/9/second`를 함께 받는 흐름이 들어 있습니다.
- README도 `/lights/\*/value` 같은 마스크 바인딩 예를 제공합니다.
- 마스크는 편하지만 범위가 넓으면 의도하지 않은 메시지까지 받을 수 있으므로 설계 규칙이 필요합니다.



비유: 주소는 택배 송장, Type tag는 상자 라벨, Arguments는 상자 안 물건입니다. 받는 쪽은 주소가 맞고 라벨이 맞을 때만 안전하게 꺼내 씁니다.

비유

마스크는 폴더 검색의 와일드카드입니다. `/lights/*/value`는 lights 폴더 아래 어떤 이름이든 value 파일이면 잡는다는 뜻입니다.

실제 예시

`/avatar/*/weight`로 smile, angry, blink 같은 표정 weight를 하나의 핸들러에서 받아 address별로 분기할 수 있습니다.

바로 해볼 것

처음에는 정확한 주소로 Bind하고, 주소 체계가 안정된 뒤 반복 패턴만 마스크로 바꿔보세요.

핵심 개념

- `OSCMatchPattern`은 메시지 안의 값 타입 순서가 기대한 구조와 맞는지 검사하는 데 사용합니다.
- 예제는 String, Int, Bool, Bool 순서가 맞으면 정상 메시지로 처리합니다.
- bool은 True 또는 False 타입으로 들어올 수 있어 패턴 검사에서 별도 고려가 들어갑니다.
- 외부 도구와 붙일 때는 주소뿐 아니라 타입까지 검증해야 디버깅 시간이 줄어듭니다.



비유: 주소는 택배 송장, Type tag는 상자 라벨, Arguments는 상자 안 물건입니다. 받는 쪽은 주소가 맞고 라벨이 맞을 때만 안전하게 꺼내 씁니다.

비유

주소가 맞는 택배라도 상자 안 물건이 계약서와 다르면 반송해야 합니다. Match Pattern은 이 검수표입니다.

실제 예시

`/camera/lens ,ff`라고 약속했으면 `focalLength`와 `aperture` 두 float가 들어왔는지 먼저 검사합니다.

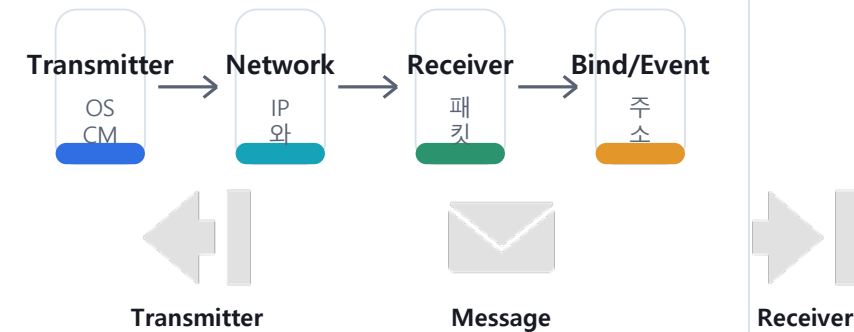
바로 해볼 것

중요한 명령 3개에는 Match Pattern을 붙이고, 실패하면 Console Warning을 남기도록 구현하세요.

핵심 개념

- extOSC의 Mapping은 들어오거나 나가는 값을 다른 범위로 바꾸는 데 사용합니다.
- `OSCUtilities.Map`은 inputMin/inputMax에서 outputMin/outputMax로 값을 변환하고 clamp 옵션을 제공합니다.
- Mapping Example은 rotate, scale, position 값을 OSC float 3개로 보내는 구조를 보여줍니다.
- 라이브 제어에서는 0-1 슬라이더 값을 Unity의 실제 물리 단위나 렌더 값으로 바꾸는 일이 매우 많습니다.

extOSC 안에서 데이터가 움직이는 길



비유

Mapping은 번역기입니다. 패널에서는 0-1로 말하지만 Unity 조명은 0-8 intensity로 듣게 만들 수 있습니다.

실제 예시

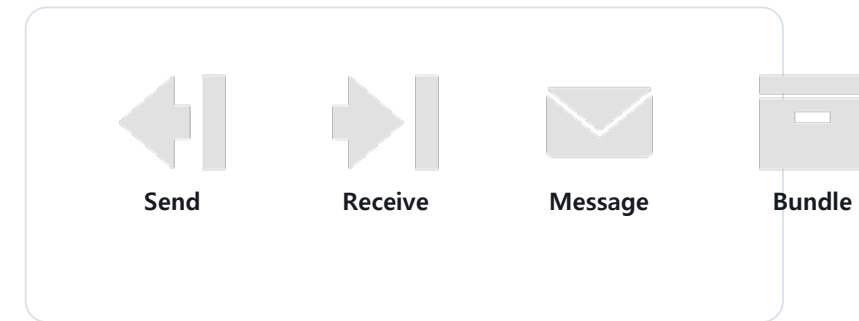
패널 slider 0.0-1.0을 `Light.intensity` 0-4, `Volume.weight` 0-1, `Camera.fieldOfView` 25-70으로 각각 매핑합니다.

바로 해볼 것

`OSCUtilities.Map(value, 0, 1, 25, 70)`로 FOV를

핵심 개념

- Receiver Event 계열은 OSC 메시지를 받았을 때 UnityEvent를 호출하는 컴포넌트입니다.
- Float Event, Int Event, Bool Event, Vector3 Event, Color Event 등 타입별 컴포넌트가 제공됩니다.
- 코드 작성 없이 Inspector에서 대상 오브젝트와 함수를 연결할 수 있습니다.
- 빠른 프로토타입에는 좋지만, 프로젝트가 커지면 주소 계약과 타입 검증을 코드로 보강하는 편이 안전합니다.



비유

Receiver Event는 초인종입니다. 특정 주소의 신호가 오면 Inspector에 연결한 함수가 바로 눌립니다.

실제 예시

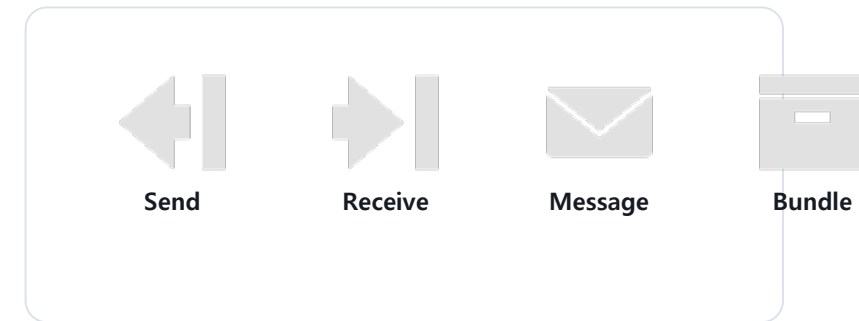
`/stage/fog/enabled` Bool Event를 받아 Fog Controller의 `SetFog(bool)` 함수를 호출합니다.

바로 해볼 것

Float Event 하나를 만들어 Light intensity를 조절하고, 같은 기능을 코드 Bind 방식으로 한 번 구현해보세요.

핵심 개념

- Informer는 Unity 오브젝트의 값 변화를 관찰하다가 OSC 메시지로 보내는 컴포넌트입니다.
- `InformOnChanged`가 켜져 있으면 값이 바뀔 때만 보냅니다.
- `InformInterval`을 쓰면 일정 간격으로 값을 보낼 수 있습니다.
- UI 컨트롤 생성 메뉴는 Slider, Button, Pad, Rotary에 맞는 Informer를 자동으로 붙이는 흐름을 지원합니다.



비유

Informer는 현황 보고자입니다. 무대 장치 값이 바뀌면 관제실에 '지금 값은 이렇다'고 알려줍니다.

실제 예시

Unity 안의 카메라 FOV 슬라이더 값이 바뀔 때마다 외부 모니터링 앱에 `/unity/camera/fov`를 보냅니다.

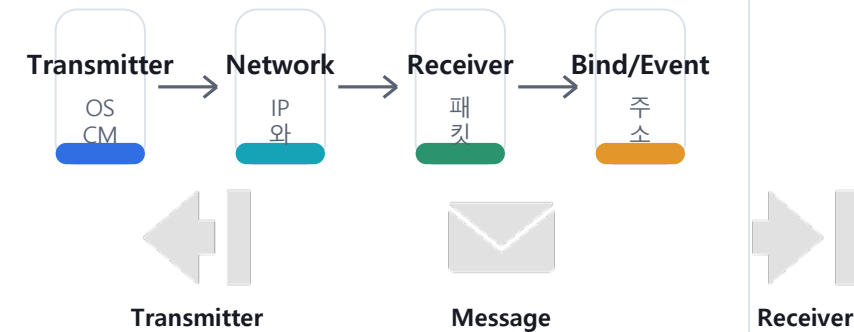
바로 해볼 것

OSC Slider를 만들고 Informer가 어떤 컴포넌트의 `value`를 ReflectionTarget으로 잡는지 Inspector에서 확인하세요.

핵심 개념

- Receiver Reflection은 받은 OSC 값을 특정 Component의 Field, Property, Method에 연결하는 계열입니다.
- 소스에는 Bool, Float, Int, Vector2, Vector3, Color, Quaternion 등 타입별 Reflection 컴포넌트가 있습니다.
- 코드 없이 빠르게 연결할 수 있지만, 리팩터링으로 멤버 이름이 바뀌면 연결이 깨질 수 있습니다.
- 포트폴리오에서는 빠른 데모는 Reflection, 핵심 시스템은 명시적 C# 코드로 구성하는 균형이 좋습니다.

extOSC 안에서 데이터가 움직이는 길



비유

Reflection은 리모컨 버튼을 기계 안쪽 스위치에 직접 테이프로 붙이는 것과 비슷합니다. 빠르지만 어디에 붙였는지 기록이 중요합니다.

실제 예시

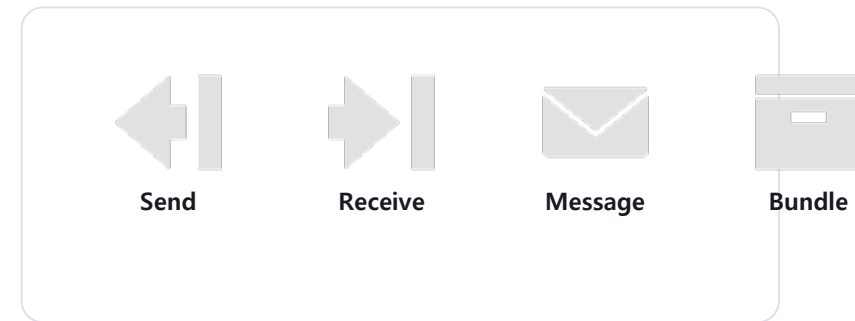
`/stage/prop/scale` Vector3 Reflection을 Transform.localScale에 연결하면 외부 값으로 프롭 크기를 바꿀 수 있습니다.

바로 해볼 것

Reflection을 쓴 곳은 README의 OSC Address Table에

핵심 개념

- GameObject > extOSC 메뉴에서 Pad, Slider, Button, Rotary, Multiply Sliders를 만들 수 있습니다.
- UI 생성 창은 control color, informer transmitter, inform address, on changed, interval 같은 설정을 함께 다룹니다.
- UI는 Unity 내부 테스트 패널이나 외부 제어 앱 프로토타입을 빠르게 만드는 데 좋습니다.
- 실무형 포트폴리오에서는 '운영자 패널' 씬을 별도로 만들어 OSC 송신과 수신 상태를 함께 보여주면 설득력이 올라갑니다.



비유

UI Controls는 임시 조종석입니다. 실제 방송 장비가 없어도 Unity 안에서 버튼과 슬라이더로 전체 제어 체계를 시험할 수 있습니다.

실제 예시

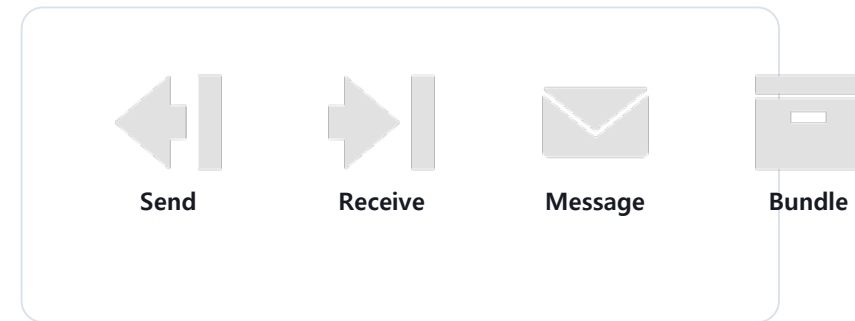
Pad는 카메라 타겟 오프셋 x/y, Rotary는 색온도, Button은 효과 트리거, Multiply Sliders는 표정 weight 묶음에 적합합니다.

바로 해볼 것

OSC Panel 씬을 만들고 Slider 3개, Button 3개, Pad 1개를 배치한 뒤 각 주소를 표로 정리하세요.

핵심 개념

- Tools > extOSC > OSC Console은 송수신 OSC 패킷을 추적하는 도구로 README와 문서에 소개됩니다.
- Getting Started 문서도 OSC Console에서 예제 메시지 동작을 확인하라고 안내합니다.
- Console은 주소, 값, 송수신 방향을 눈으로 확인하는 첫 번째 디버깅 장소입니다.
- 포트폴리오 영상에서는 Console을 잠깐 보여주면 '실제로 OSC가 흐른다'는 증거가 됩니다.



비유

OSC Console은 물류 추적 화면입니다. 메시지가 보냈는지, 받았는지, 어떤 주소였는지 확인합니다.

실제 예시

`/example/1` Hello world 메시지가 찍히면 기본 송수신 연결은 성공입니다.

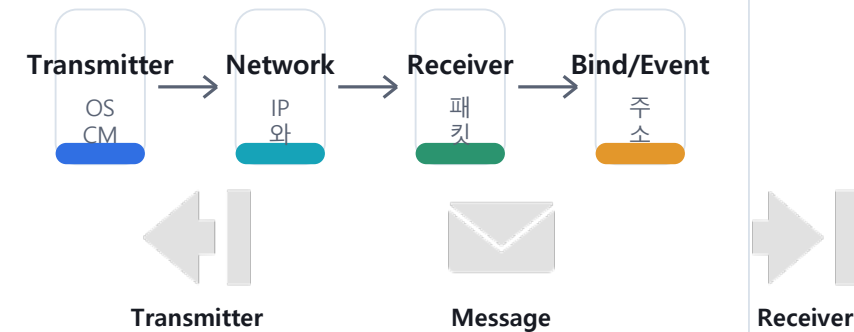
바로 해볼 것

실습 때는 Game View만 보지 말고 OSC Console을 옆에 띄워 주소와 값이 맞는지 같이 확인하세요.

핵심 개념

- Tools > extOSC > OSC Debug는 OSC 패킷 디버깅용 도구로 README에 소개됩니다.
- Tools > extOSC > OSC Mapping은 맵핑 설정을 다루는 에디터 창입니다.
- 패키지 메뉴에는 GitHub Repository, Roadmap, Wiki, Unity Forum 링크도 포함되어 있습니다.
- Tools > extOSC > Settings에는 문자열 인코딩 ASCII/UTF8 선택과 Receiver Drown 감지 토글이 있습니다.

extOSC 안에서 데이터가 움직이는 길



비유

Console이 CCTV라면 Debug는 테스트 송수신 장비, Mapping Window는 변환 규칙을 편집하는 패치 베이입니다.

실제 예시

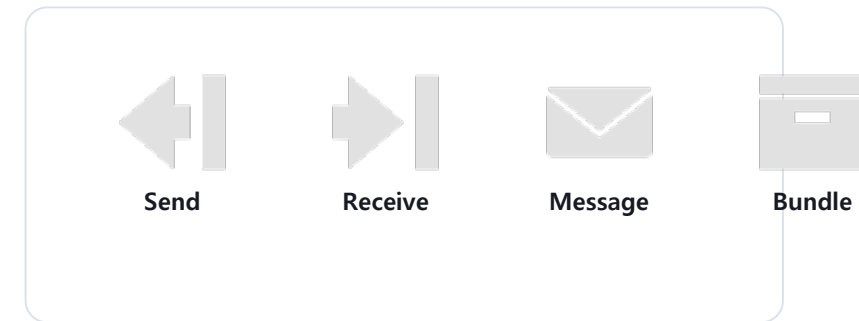
외부 앱 없이 Unity 안에서 메시지를 보내 보고, 수신 쪽 반응과 Mapping 결과를 확인합니다.

바로 해볼 것

Tools > extOSC 메뉴를 실제로 열어보고, 자료의 메뉴

핵심 개념

- extOSC에는 Ping Client와 Ping Server 컴포넌트가 있습니다.
- Ping Client는 Interval마다 메시지를 보내고, Timeout 안에 응답을 받았는지 `IsAvailable`로 판단합니다.
- 문서 PDF는 Ping이 앱이 아직 실행 중인지 감시하는 데 유용하다고 설명합니다.
- 라이브 운영에서는 연결 상태 표시등을 UI에 두는 것이 좋습니다.



비유

Ping은 '들리세요?' 확인입니다. 상대가 답하면 초록불, 일정 시간 답이 없으면 빨간불을 켭니다.

실제 예시

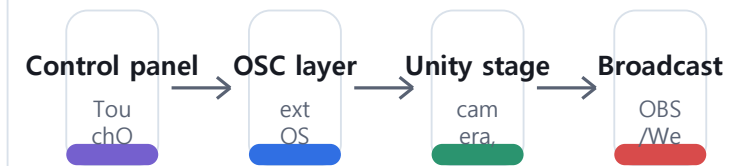
운영자 패널에서 Unity Stage가 살아 있는지 `/ping`으로 확인하고, 2초 이상 응답이 없으면 'Disconnected'를 표시합니다.

바로 해볼 것

Ping 샘플을 Import하고 Interval과 Timeout 값을 바꾸며 상태 표시가 어떻게 변하는지 관찰하세요.

핵심 개념

- extOSC 1.20.0 changelog에는 Receiver drown detection 추가가 기록되어 있습니다.
- 설치된 `OSCReceiver.cs`는 Update 처리 중 일정 시간 이상 패킷 처리가 길어질 때 감지하는 구조를 포함합니다.
- 이는 프레임 안에서 너무 많은 OSC 패킷을 처리하다가 Unity 메인 스레드가 밀리는 상황을 조심하라는 신호로 볼 수 있습니다.
- 해결 방향은 전송 빈도 제한, 값 변화 시에만 전송, Bundle 사용, 주소 필터링, 불필요 메시지 폐기입니다.



주의

이 구조는 포트폴리오 설계 제안입니다. 특정 기업이 extOSC나 VMC, OBS WebSocket을 사용한다고 단정하지 않습니다.

비유

Drown은 우편함에 편지가 너무 많이 쌓여 담당자가 업무 시간 안에 분류하지 못하는 상황입니다.

실제 예시

표정 weight 50개를 매 프레임 모두 보내기보다 바뀐 값만 보내거나 20Hz로 제한하면 처리 부담이 줄어듭니다.

바로 해볼 것

Profiler와 OSC Console을 같이 켜고 메시지 빈도를 10Hz, 30Hz, 60Hz로 바꿔 CPU 사용과 반응성을

핵심 개념

- `127.0.0.1`은 같은 컴퓨터 안에서 테스트할 때 쓰는 localhost 주소입니다.
- 다른 기기에서 Unity로 보내려면 Unity가 실행 중인 PC의 LAN IPv4 주소를 RemoteHost로 넣어야 합니다.
- 송신 쪽 RemotePort와 수신 쪽 LocalPort가 같아야 합니다.
- 포트 충돌, 방화벽 차단, IP 오타, 같은 포트 중복 사용이 가장 흔한 문제입니다.



비유: 주소는 택배 송장, Type tag는 상자 라벨, Arguments는 상자 안 물건입니다. 받는 쪽은 주소가 맞고 라벨이 맞을 때만 안전하게 꺼내 씁니다.

비유

IP는 건물 주소, Port는 방 번호입니다. 건물은 맞아도 방 번호가 틀리면 메시지가 도착하지 않습니다.

실제 예시

태블릿 TouchOSC -> Unity PC라면 태블릿의 Host를 PC IPv4로, Port를 Unity OSCReceiver LocalPort로 맞춥니다.

바로 해볼 것

Windows 터미널에서 `ipconfig`로 IPv4를 확인하고, Unity LocalPort와 외부 앱 Target Port를 표로 적으세요.

핵심 개념

- extOSC 메뉴에는 OSCValue.String Encoding을 ASCII 또는 UTF8로 바꾸는 설정이 있습니다.
- VMC Protocol은 UTF-8 사용을 명시합니다. 데이터에 비 ASCII 문자가 포함될 수 있기 때문입니다.
- 한국어 캐릭터 이름, 프리셋 이름, 씬 이름을 OSC 문자열로 보낼 계획이면 UTF8 설정을 확인해야 합니다.
- 서로 다른 앱이 문자열을 주고받을 때는 인코딩을 주소 계약서에 적어두는 것이 좋습니다.



비유: 주소는 택배 송장, Type tag는 상자 라벨, Arguments는 상자 안 물건입니다. 받는 쪽은 주소가 맞고 라벨이 맞을 때만 안전하게 꺼내 씁니다.

비유

인코딩은 언어 사전입니다. 보내는 쪽과 받는 쪽이 같은 사전을 펴야 글자가 깨지지 않습니다.

실제 예시

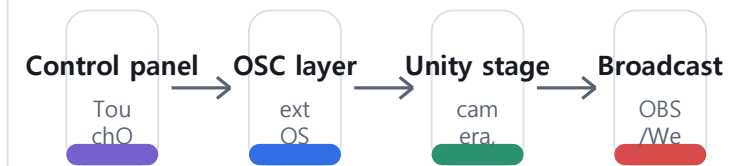
``/preset/name "여름무대"``` 같은 문자열을 보내려면 수신 앱이 UTF-8을 기대하는지 확인합니다.

바로 해볼 것

영문 문자열과 한국어 문자열을 각각 보내보고 수신 로그가 깨지지 않는지 테스트하세요.

핵심 개념

- VMC Protocol은 OSC over UDP/IP를 사용한다고 명시합니다.
- 일반적으로 Marionette는 port 39539, Performer는 39539로 전송하고 Assistant용 server는 39540을 흔히 사용한다고 설명합니다.
- VMC는 bone, blendshape, camera 등 라이브 아바타 관련 데이터를 다루는 OSC 기반 프로토콜입니다.
- extOSC는 VMC 전용 구현체가 아니라 OSC 송수신 도구입니다. VMC 주소와 타입 규약은 별도로 맞춰야 합니다.



주의

이 구조는 포트폴리오 설계 제안입니다. 특정 기업이 extOSC나 VMC, OBS WebSocket을 사용한다고 단정하지 않습니다.

비유

OSC가 도로라면 VMC는 그 도로 위에서 쓰는 배송 양식입니다. extOSC는 Unity에서 도로에 진입하는 차량입니다.

실제 예시

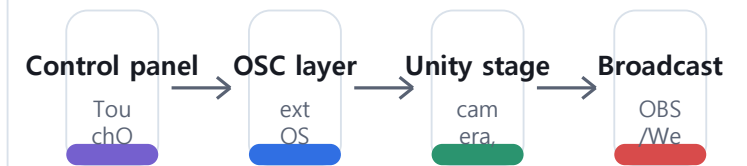
VMC의 `/VMC/Ext/Bone/Pos` 계열 메시지를 받으려면 해당 주소와 값 순서를 VMC 문서에 맞춰 파싱해야 합니다.

바로 해볼 것

VMC 연동을 목표로 한다면 먼저 포트 39539 수신 테스트를 만들고, 실제 파싱은 메시지별로 작은 단위

핵심 개념

- OBS Studio 28 이상은 WebSocket 시스템이 기본 포함되어 외부 도구로 씬과 소스를 자동화하거나 제어할 수 있습니다.
- OBS는 OSC가 아니라 WebSocket 기반 제어가 공식 안내의 중심입니다.
- 따라서 Unity OSC -> 브릿지 앱 -> OBS WebSocket 같은 구조로 연결하는 것이 자연스럽습니다.
- 보안상 OBS WebSocket 인증과 비밀번호 설정을 확인해야 합니다.



주의

이 구조는 포트폴리오 설계 제안입니다. 특정 기업이 extOSC나 VMC, OBS WebSocket을 사용한다고 단정하지 않습니다.

비유

Unity와 외부 패널은 OSC 무전기를 쓰고, OBS는 WebSocket 전화기를 씁니다. 브릿지는 두 장비 사이의 통역사입니다.

실제 예시

Unity가 `/broadcast/scene 2`를 받으면 내부 상태를 바꾸고, 별도 브릿지가 OBS에 Scene 전환 요청을 보냅니다.

바로 해볼 것

포트폴리오 1차 버전에서는 OBS 직접 제어보다 Unity 내부 씬 전환과 로그 표시를 먼저 완성하세요.

아래 표는 설치된 `OSCValueType` enum과 `OSCValue` 생성 메서드를 기준으로 정리했습니다. 자주 쓰는 것은 Float, Int, Bool, String, Vector 계열을 구성하는 여러 Float입니다.

타입	태그	생성 예	주요 용도
Int	i	OSCValue.Int(3)	프리셋 번호, 모드 번호
Float	f	OSCValue.Float(0.75f)	슬라이더, weight, intensity
Bool	T/F	OSCValue.Bool(true)	토글, on/off 상태
String	s	OSCValue.String("cutA")	프리셋명, 상태명
Blob	b	OSCValue.Blob(bytes)	바이너리 구조, marshalling
Color	r	OSCValue.Color(Color.red)	조명/패널 색
TimeTag	t	OSCValue.TimeTag(dt)	시간 정보
Midi	m	OSCValue.Midi(midi)	MIDI 호환 데이터
Array	[]	OSCValue.Array(...)	여러 값을 중첩 묶음

Getting Started 샘플의 핵심은 Address 하나와 값 하나를 메시지에 넣고 Transmitter로 보내는 것입니다.

C# example

```
using extOSC;
using UnityEngine;

public class SendHello : MonoBehaviour
{
    public OSCTransmitter transmitter;

    private void Start()
    {
        var message = new OSCMessage("/example/1");
        message.AddValue(OSCValue.String("Hello, world!"));
        transmitter.Send(message);
    }
}
```

읽는 법

- `/example/1`은 편지의 주소입니다.
- `OSCValue.String`은 메시지 인자 하나를 string 타입으로 넣습니다.
- `Send` 호출 시 RemoteHost와 RemotePort로 UDP 전송됩니다.
- 이 구조를 `/stage/light/key/intensity` 같은 프로젝트 주소로 바꾸면 바로 응용됩니다.

Receiver는 `Bind`로 주소와 메서드를 연결합니다. 메시지가 들어오면 extOSC가 주소를 비교한 뒤 콜백을 호출합니다.

C# example

```
using extOSC;
using UnityEngine;

public class ReceiveHello : MonoBehaviour
{
    public OSCReceiver receiver;

    private void Start()
    {
        receiver.Bind("/example/1", OnMessage);
    }

    private void OnMessage(OSCMessage message)
    {
        Debug.Log($"Received: {message}");
    }
}
```

읽는 법

- 수신 포트는 Receiver의 LocalPort와 외부 송신 앱의 Target Port가 같아야 합니다.
- 콜백 안에서는 먼저 타입을 확인한 다음 Unity 오브젝트를 제어합니다.
- 주소가 맞아도 값 타입이 틀릴 수 있으니 중요한 명령에는 검사 코드를 넣습니다.

실무형 포트폴리오에서는 '메시지를 받았다'에서 끝내지 말고 실제 씬의 렌더 요소가 바뀌어야 합니다.

C# example

```
using extOSC;
using UnityEngine;

public class OscLightController : MonoBehaviour
{
    public OSCReceiver receiver;
    public Light keyLight;

    private const string Address = "/stage/light/key/intensity";

    private void Start()
    {
        receiver.Bind(Address, OnIntensity);
    }

    private void OnIntensity(OSCMessage message)
    {
        if (!message.ToFloat(out var value)) return;
        keyLight.intensity = Mathf.Clamp(value, 0f, 8f);
    }
}
```

읽는 법

- 주소는 명확하게 도메인/대상/속성 순서로 구성합니다.
- 외부 슬라이더가 0-1만 보낸다면 Mapping으로 0-8 범위로 바꿀 수 있습니다.
- Clamp를 넣으면 잘못된 외부 값이 렌더 상태를 망가뜨리는 일을 줄입니다.

VRM/VMC 여부와 무관하게, 포트폴리오에서는 '아바타 상태를 외부 신호로 제어한다'는 흐름을 먼저 만들 수 있습니다.

C# example

```
using extOSC;
using UnityEngine;

public class OscExpressionRouter : MonoBehaviour
{
    public OSCReceiver receiver;
    public Animator animator;

    private void Start()
    {
        receiver.Bind("/avatar/expression/*/weight", OnExpression);
    }

    private void OnExpression(OSCMessage message)
    {
        if (!message.ToFloat(out var weight)) return;

        // Example address: /avatar/expression/smile/weight
        var parts = message.Address.Split('/');
        if (parts.Length < 4) return;

        var expressionName = parts[3];
        animator.SetFloat(expressionName, Mathf.Clamp01(weight));
    }
}
```

읽는 법

- 마스크 주소를 쓰면 표정 이름을 address에서 꺼내 공통 처리할 수 있습니다.
- 실제 VRM Expression 적용은 사용 중인 VRM 런타임 API에 맞게 연결해야 합니다.
- 이 코드는 구조 예시입니다. 특정 기업의 실제 아바타 파이프라인을 단정하지 않습니다.

여러 상태를 한 번에 보내야 할 때는 Bundle 개념을 이해해야 합니다. extOSC Transmitter의 UseBundle도 이 맥락에서 봅니다.

C# example

```
using extOSC;
using UnityEngine;

public class SendStagePreset : MonoBehaviour
{
    public OSCTransmitter transmitter;

    public void SendPreset()
    {
        var bundle = new OSCBundle();
        bundle.AddPacket(OSCMessage.Create(
            "/stage/light/key/intensity", OSCValue.Float(3.5f)));
        bundle.AddPacket(OSCMessage.Create(
            "/stage/fog/density", OSCValue.Float(0.25f)));
        bundle.AddPacket(OSCMessage.Create(
            "/camera/fov", OSCValue.Float(35f)));

        transmitter.Send(bundle);
    }
}
```

읽는 법

- Bundle은 여러 메시지를 하나로 묶는 컨테이너입니다.
- 수신 쪽은 Bundle 안의 각 Message를 다시 순회해 처리합니다.
- 무대 프리셋처럼 여러 값이 한 타이밍에 바뀌는 경우에 자연스럽게.

OSC 주소는 코드보다 먼저 설계하는 편이 좋습니다. 주소가 안정되면 UI, 스크립트, 문서, 테스트가 모두 같은 언어를 씁니다.

규칙	좋은 예	피해야 할 예	이유
도메인부터 시작	/stage/light/key/intensity	/intensity1	프로젝트가 커져도 찾기 쉽습니다.
명사/속성 구분	/camera/main/fov	/cameraFov	마스크와 문서화가 쉬워집니다.
타입 고정	/avatar/expression/smile/weight -> float	가끔 string, 가끔 float	수신 코드가 단순해집니다.
트리거와 상태 분리	/effect/fire/trigger	/effect/fire/enabled, /effect/fire 1	순간 명령과 지속 상태는 다릅니다.
버전 관리	/v1/stage/preset/load	/stage/load	외부 앱과 계약 변경을 관리하기 쉽습니다.

아래 표처럼 README에 OSC Address Table을 넣으면 단순 데모가 아니라 협업 가능한 시스템처럼 보입니다.

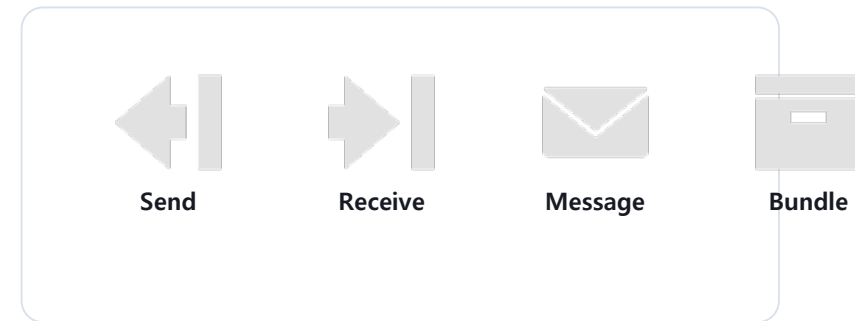
Address	Type	Range	Unity target
/stage/light/key/intensity	float	0-8	Key Light.intensity
/stage/light/key/color	float,float,float	0-1 RGB	Key Light.color
/camera/main/fov	float	25-70	Cinemachine/Camera FOV
/avatar/expression/*/weight	float	0-1	Expression weight router
/effect/*/trigger	impulse	none	Particle/VFX trigger
/stage/preset/load	string	preset id	StagePresetController
/system/ping	impulse	none	Connection monitor

이 순서대로 만들면 extOSC 학습이 곧 포트폴리오 기능으로 쌓입니다.

단계	목표	완성 기준	보여줄 역량
1	Hello OSC	Console에 송수신 로그 표시	설치/포트/주소 기본기
2	Light Panel	슬라이더로 조명 밝기/색 제어	실시간 렌더 파라미터 제어
3	Camera Control	FOV, Dutch, target offset 제어	방송 연출 제어
4	Expression Router	마스크 주소로 표정 weight 라우팅	주소 설계와 메시지 파싱
5	Preset Bundle	무대 프리셋을 Bundle로 전환	상태 묶음과 운영성
6	Debug Dashboard	Ping, last message, dropped/error count 표시	실무 안정성

핵심 개념

- 목표는 외부 앱 없이 Unity 내부에서 Transmitter와 Receiver를 같은 포트에 연결해 메시지가 오는 것을 확인하는 것입니다.
- Receiver LocalPort는 7001, Transmitter RemoteHost는 127.0.0.1, RemotePort는 7001로 둡니다.
- 주소는 `/study/hello`, 값은 string `Hello extOSC`로 시작합니다.
- 성공 기준은 OSC Console과 Unity Console 둘 다에서 메시지를 확인하는 것입니다.



비유

한 컴퓨터 안에서 자기 자신에게 편지를 보내는 연습입니다. 우체국이 실제로 동작하는지 확인하는 가장 작은 실험입니다.

실제 예시

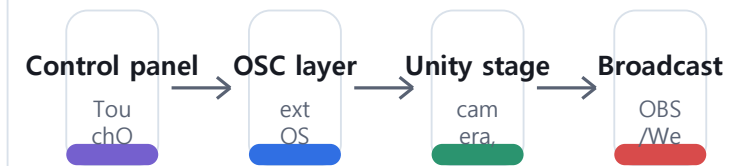
SimpleMessageReceiver와 SimpleMessageTransmitter 샘플을 복사해 Address만 `/study/hello`로 바꿉니다.

바로 해볼 것

이 실습이 실패하면 이후 기능을 만들지 말고 IP, Port, Receiver 활성화부터 확인하세요.

핵심 개념

- URP 포트폴리오와 연결하려면 OSC로 렌더 파라미터를 바꿔야 합니다.
- 조명 intensity, Volume weight, Bloom intensity, Fog density, Camera FOV가 좋은 첫 대상입니다.
- 외부 입력은 0-1로 받고 Unity 내부 값은 Mapping으로 변환합니다.
- Before/After 화면과 Frame Debugger/Profiler 캡처를 함께 남기면 렌더 최적화 포트폴리오와 연결됩니다.



주의

이 구조는 포트폴리오 설계 제안입니다. 특정 기업이 extOSC나 VMC, OBS WebSocket을 사용한다고 단정하지 않습니다.

비유

OSC는 조명 콘솔의 페이더, Unity는 실제 무대 조명입니다. 페이더 값 0-1을 실제 조명 범위로 번역해야 합니다.

실제 예시

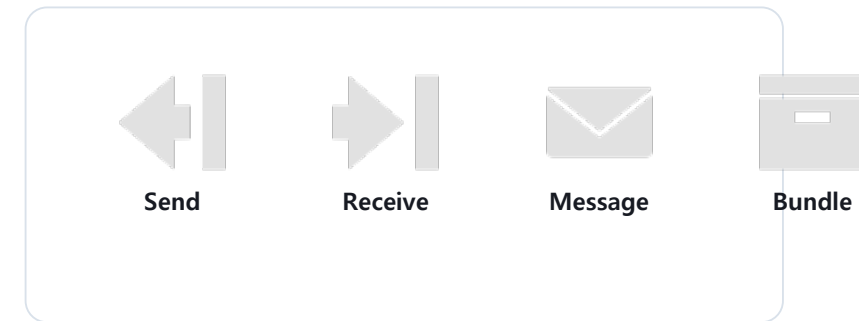
`/render/bloom/intensity 0.6`을 받으면 URP Volume Override의 bloom intensity를 바꿉니다.

바로 해볼 것

렌더 스케일, SSAO, Shadow 옵션을 OSC로 켜고 끄는 실험 UI를 만들어보세요.

핵심 개념

- GameObject > extOSC > Slider/Button/Pad/Rotary로 Unity 내부 제어 패널을 만듭니다.
- 각 UI는 Informer로 메시지를 보낼 수 있고, Receiver 쪽 씬은 동일 주소를 받아 상태를 바꿉니다.
- 패널에는 연결 상태, 마지막 수신 주소, 마지막 수신 시간, 현재 프리셋을 표시합니다.
- 포트폴리오 영상에서는 패널 조작 -> Unity 씬 반응 -> OSC Console 로그를 한 화면에 잡으면 좋습니다.



비유

운영자 패널은 방송 부스의 작은 믹서입니다. 버튼 하나가 무대 전체 큐로 이어지도록 구성합니다.

실제 예시

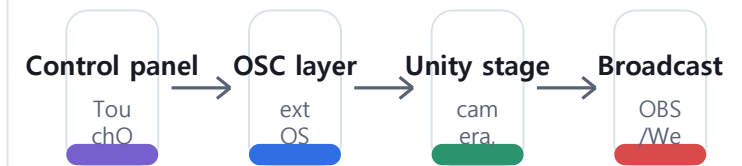
Button: ``/effect/confetti/trigger``, Slider: ``/camera/main/fov``, Pad: ``/camera/target/offset``.

바로 해볼 것

UI가 예쁘기보다 주소가 명확하고 반응이 안정적인지 먼저 확인하세요.

핵심 개념

- VMC Protocol은 OSC over UDP/IP와 UTF-8 사용을 명시합니다.
- VMC 메시지는 주소와 타입이 정해진 별도 규약이므로 extOSC의 일반 Bind만으로 끝나지 않고 파싱 코드가 필요합니다.
- 처음에는 모든 VMC 메시지를 구현하지 말고, 한 메시지 주소를 받아 로그로 구조를 확인하는 것부터 시작합니다.
- 필요 없는 메시지는 버리고, 주기와 부하를 측정하는 습관이 중요합니다.



주의

이 구조는 포트폴리오 설계 제안입니다. 특정 기업이 extOSC나 VMC, OBS WebSocket을 사용한다고 단정하지 않습니다.

비유

VMC는 다른 나라의 업무 양식입니다. extOSC가 편지를 배달해도, 양식 해석은 우리가 해야 합니다.

실제 예시

포트 39539로 Receiver를 열고 `/VMC/...` 주소가 들어오는지 Console에 찍어봅니다.

바로 해볼 것

실제 아바타 적용 전에 메시지 주소, 값 개수, 타입을 CSV나 README 표로 정리하세요.

핵심 개념

- 메시지가 안 오면 IP, Port, LocalHostMode, 방화벽, 같은 포트 사용 여부를 먼저 봅니다.
- 값이 이상하면 address typo, type mismatch, 0-1과 실제 단위 변환 문제를 확인합니다.
- 너무 버벅이면 메시지 빈도, Bundle 사용 여부, 불필요한 로그, Receiver Drown 경고를 확인합니다.
- 문자열이 깨지면 ASCII/UTF8 설정과 외부 앱 인코딩을 확인합니다.



비유: 주소는 택배 송장, Type tag는 상자 라벨, Arguments는 상자 안 물건입니다. 받는 쪽은 주소가 맞고 라벨이 맞을 때만 안전하게 꺼내 씁니다.

비유

문제 해결은 전기 배선 점검과 비슷합니다. 전원이 들어오는지, 선이 맞는지, 장비가 과부하인지 순서대로 봅니다.

실제 예시

포트 충돌 시 extOSC 백엔드는 같은 포트를 다른 앱이 사용 중이라는 Socket Error를 Console에 출력할 수 있습니다.

바로 해볼 것

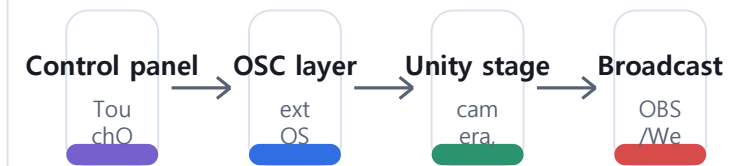
문제가 생기면 Game View보다 Console, OSC Console, Profiler를 먼저 열어보세요.

기능을 만들 때마다 아래 항목을 통과시키면 포트폴리오 완성도가 확 올라갑니다.

항목	확인 방법	통과 기준	자료화
설치	manifest/lock 확인	버전과 hash 기록	README 환경 섹션
송수신	OSC Console	주소와 값이 예상대로 표시	캡처 이미지
타입	Match Pattern/ToFloat 등	잘못된 타입 무시	테스트 표
성능	Profiler	메시지 폭주 시 프레임 방어	비교 그래프
운영	Ping/상태 UI	연결 끊김 표시	데모 영상
문서	Address Table	외부 앱이 보고 따라 가능	README/PDF

핵심 개념

- Scene 1: OSC Basics - 송수신 로그와 간단한 큐브 제어.
- Scene 2: Live Stage Control - 조명, 카메라, Volume, 이펙트 제어.
- Scene 3: Avatar Control Mock - 표정/포즈/상태 라우터.
- Scene 4: Debug Dashboard - Ping, 마지막 메시지, 주소 테이블, 성능 수치.
- 문서: 설치법, 주소 계약서, 트러블슈팅, 성능 측정 결과, 데모 영상 링크.



주의

이 구조는 포트폴리오 설계 제안입니다. 특정 기업이 extOSC나 VMC, OBS WebSocket을 사용한다고 단정하지 않습니다.

비유

포트폴리오는 완성된 게임보다 '실시간 운영 도구를 만드는 사람'이라는 증거물입니다.

실제 예시

면접에서는 'OSC가 뭐냐'보다 '외부 도구가 보낸 불안정한 값을 어떻게 안전하게 Unity에 반영했냐'를 설명하는 쪽이 강합니다.

바로 해볼 것

각 씬마다 30초짜리 데모 영상을 따로 찍고, 마지막에 전체 시스템 흐름 1분 영상을 만드세요.

단순히 'OSC 해봤습니다'보다 아래처럼 설계와 운영 관점으로 말하면 개발자 역량이 더 잘 보입니다.

질문	좋은 답변 키워드	보여줄 산출물
OSC가 뭔가요?	주소 기반 실시간 제어 메시지, Message/Bundle	메시지 구조 다이어그램
왜 extOSC인가요?	Unity 컴포넌트, 코드 API, Console/Debug, Samples	패키지 구조 캡처
어떻게 안전하게 받나요?	타입 검사, clamp, 마스크 범위 제한	수신 코드
성능은요?	빈도 제한, UseBundle, Drown 감지, Profiler	측정 표
실무 응용은요?	운영자 패널, VMC 파싱, OBS 브릿지 분리	아키텍처 그림

이 자료는 공식 웹 문서와 현재 프로젝트에 설치된 로컬 패키지 파일을 함께 근거로 작성했습니다. 특정 기업이 실제로 extOSC, VMC, VRM, OBS WebSocket을 쓴다는 의미는 아니며, 개발자 포트폴리오에서 보여줄 수 있는 실시간 제어 역량의 예시로 구분했습니다.

사실로 쓴 것

OSC 스펙 구조, extOSC README/패키지 manifest/소스 코드, Unity Package Manager 문서, VMC Protocol, OBS WebSocket 안내처럼 출처에서 확인 가능한 내용입니다.

제안으로 쓴 것

운영자 패널, 라이브 스튜디오 포트폴리오, 주소 설계 예시, VMC/OBS 브릿지 구조는 사용자의 프로젝트 목표에 맞춘 설계 제안입니다.

로컬 패키지 기준 주의

`extOSC - Documentation.pdf`에는 과거 설명이 포함되어 있어 현재 README/package.json/source/changelog와 다른 부분이 있습니다. 이 자료는 설치된 1.21.0 패키지 파일을 우선 기준으로 삼았습니다.

S1

OpenSoundControl Specification 1.0

OpenSoundControl.org / Matt Wright

https://opensoundcontrol.stanford.edu/spec-1_0.html

S2

extOSC GitHub README

Iam1337 / dr. ext

<https://github.com/Iam1337/extOSC>

S3

OpenUPM extOSC package page

OpenUPM

<https://openupm.com/packages/com.iam1337.extosc/>

S4

Unity Manual - Install a UPM package from a Git URL

Unity Technologies

<https://docs.unity3d.com/Manual/upm-ui-giturl.html>

S5

Unity Manual - Create samples for your package

Unity Technologies

<https://docs.unity3d.com/Manual/cus-samples.html>

S6

VMC Protocol specification

VirtualMotionCaptureProtocol

<https://protocol.vmc.info/english.html>

S7

OBS Remote Control Guide

OBS Project

<https://obsproject.com/kb/remote-control-guide>

L1

Project package manifest and lock file

Local project

[Packages/manifest.json](#), [Packages/packages-lock.json](#)

L2

Installed extOSC package manifest

Local package cache

[Library/PackageCache/com.iam1337.extosc@b7c2bfa81633/package.json](#)

L3

Installed extOSC README and changelog

Local package cache

[Library/PackageCache/com.iam1337.extosc@b7c2bfa81633/README.md](#), [CHANGELOG.md](#)

L4

Installed extOSC source scripts

Local package cache

[OSCReceiver.cs](#), [OSCTransmitter.cs](#), [OSCMessages.cs](#), [OSCValue.cs](#), [OSCUtilities.cs](#)

L5

Installed extOSC examples and documentation PDF

Local package cache

[Examples~/](#), [extOSC - Documentation.pdf](#)

프로젝트에서 다시 확인할 위치

Unity Package Manager > extOSC > Samples, Tools > extOSC 메뉴, GameObject > extOSC 메뉴, 그리고 Library/PackageCache의 source scripts를 함께 확인하면 PDF 내용과 실제 프로젝트 상태를 대조할 수 있습니다.